

目录

1	工业轴承设备剩余寿命预测系统的实现项目管理计划文档	1
1.1	文档信息	1
1.2	版本记录	1
1.3	1. 项目概述	1
1.4	2. 项目目标	2
1.4.1	2.1 交付目标	2
1.4.2	2.2 管理目标	2
1.5	3. 项目组织结构	2
1.6	4. 人员分工	2
1.7	5. 进度管理计划	2
1.8	6. 资源管理计划	2
1.8.1	6.1 人力资源	2
1.8.2	6.2 技术资源	3
1.8.3	6.3 数据资源	3
1.9	7. 风险管理计划	3
1.10	8. 质量管理计划	3
1.11	9. 沟通管理计划	3
1.11.1	9.1 沟通机制	3
1.11.2	9.2 沟通内容	3
1.12	10. 配置管理计划	3
1.13	11. 文档管理规范	4
1.14	12. 项目里程碑	4
1.15	13. 结论	4

1 工业轴承设备剩余寿命预测系统的实现项目管理计划文档

1.1 文档信息

项目	内容
文档名称	项目管理计划文档
文档阶段	proposal
项目名称	工业轴承设备剩余寿命预测系统的实现
版本	V2.0
编写日期	2026-03-12
文档负责人	zyj
参与编写	cyj、zdh、zy
管理基线	8 周迭代、Git 管理、uv 环境、文档先行

1.2 版本记录

版本	日期	编写人	说明
V1.0	2025-11-09	zyj	完成项目管理骨架
V2.0	2026-03-12	zyj、cyj、zdh、zy	补充风险、质量、沟通和配置管理策略

1.3 1. 项目概述

本项目围绕“工业轴承设备剩余寿命预测系统的实现”展开，目标是在课程周期内完成一个具备真实数据接入、多模型训练、实验记录和结果展示能力的预测软件系统，并同步输出完整课程文档。项目采用小团队协作模式，强调计划可执行、分工可衡量、成果可验收。

1.4 2. 项目目标

1.4.1 2.1 交付目标

1. 交付一套可运行的 Python 工程。
2. 支持 XJTU-SY 与 PHM2012 / FEMTO 数据接入。
3. 支持多模型、多指标、多图表输出。
4. 提交完整课程文档与 PDF 材料。

1.4.2 2.2 管理目标

1. 保持项目名称、模块划分、时间安排和术语一致。
2. 保证 4 名成员分工均衡且边界清晰。
3. 通过阶段检查降低后期返工风险。

1.5 3. 项目组织结构

项目组织采用扁平协作模式，由项目负责人协调整体进度，其余成员按主责方向推进并进行交叉评审。

角色	成员	管理职责
项目负责人	zyj	里程碑管理、需求整合、架构协调、最终提交把关
数据负责人	cyj	数据资源管理、数据接口一致性、预处理链路控制
分析负责人	zdh	阶段划分、评价口径、确认测试标准制定
展示负责人	zy	可视化方案、交付材料排版、演示效果控制

1.6 4. 人员分工

为满足“分工合理、平均”的要求，项目工作量按四条主线平均分配，每人承担约 25% 的主责工作量，同时参与联调和审阅。

成员	主责工作	协作工作
zyj	架构、训练器、接口、主程序、进度管理	需求审阅、最终集成、答辩统筹
cyj	数据集接入、预处理、特征工程、数据测试	参数整理、数据说明文档
zdh	阶段策略、生存分析、指标体系、确认测试	需求校验、测试结论审阅
zy	可视化、结果输出、图表规范、演示材料	文档排版、集成验证、展示优化

1.7 5. 进度管理计划

项目总周期为 8 周，采用每周检查、每两周形成阶段成果的方式推进。

周次	计划内容	里程碑
第 1 周	开题、技术预研、数据集调研	完成开题报告和技术预研初稿
第 2 周	需求分析	完成需求定义和 SRS 初稿
第 3 周	总体设计、编码规范制定	完成设计框架和编码规范
第 4 周	数据接入与预处理	完成数据加载与基础预处理
第 5 周	特征工程与标签构造	完成阶段划分和单元测试准备
第 6 周	模型训练与模块联调	完成集成测试计划和训练链路
第 7 周	可视化、实验记录、确认测试	完成结果展示和验收验证
第 8 周	文档收尾与答辩准备	完成 PDF 导出和最终检查

1.8 6. 资源管理计划

1.8.1 6.1 人力资源

团队共 4 人，不另设专职测试或文档岗位，采用“主责 + 交叉评审”的资源使用策略，降低信息割裂问题。

1.8.2 6.2 技术资源

1. 代码托管: GitHub
2. 环境管理: uv
3. 文档编写: Markdown
4. 文档导出: Pandoc + XeLaTeX
5. 真实数据: XJTU-SY、PHM2012 / FEMTO

1.8.3 6.3 数据资源

数据资源包括官方原始压缩包、本地解压目录、实验中间结果和图表产物。原始大文件通过 Git LFS 管理，避免普通 Git 仓库过度膨胀。

1.9 7. 风险管理计划

风险编号	风险描述	概率	影响	应对措施
R1	官方数据下载入口不稳定	中	高	保留多入口和本地目录加载方式
R2	多模型实现导致接口不统一	中	高	先定义统一 API，再接入具体模型
R3	文档和实现不同步	中	高	统一以需求和设计文档为基线，变更后同步修订
R4	训练时间过长	中	中	先做合成数据和小样本验证，再跑真实实验
R5	成员进度失衡	低	中	采用每周同步和阶段回顾，及时调整任务

1.10 8. 质量管理计划

项目质量管理重点包括：

1. 需求质量：需求必须可描述、可实现、可验证。
2. 代码质量：遵守统一编码规范，避免临时脚本式开发。
3. 测试质量：单元测试、集成测试、确认测试三层覆盖关键链路。
4. 文档质量：所有课程文档术语一致、结构清晰、可直接导出 PDF。

1.11 9. 沟通管理计划

1.11.1 9.1 沟通机制

1. 每周至少进行一次进度同步。
2. 关键接口变化需在组内同步后再落代码。
3. 文档定稿前至少完成一次交叉审阅。

1.11.2 9.2 沟通内容

沟通对象	频率	主要内容
全体成员	每周	进度、风险、接口变更、下周计划
负责人到成员	按需	任务拆解、时间协调、问题跟进
文档负责人到全员	定稿前	文档一致性和提交要求确认

1.12 10. 配置管理计划

1. 所有代码与文档统一纳入 Git 管理。
2. 原始大数据文件通过 Git LFS 管理。
3. 重要分支按成员和主线建立，提交信息采用规范化格式。
4. 依赖版本通过 `pyproject.toml` 和 `uv.lock` 固定。

1.13 11. 文档管理规范

1. 提交型课程文档归档在 docx/proposal 和 docx/mid-term 下。
2. Markdown 正文放在对应阶段的 md/ 目录中。
3. PDF 放在对应阶段的父目录中。
4. 其他说明性技术文档保留在 docs/ 目录。

1.14 12. 项目里程碑

里程碑	时间	完成标准
M1 开题完成	第 1 周结束	开题报告、技术预研报告形成
M2 需求冻结	第 2 周结束	需求定义和 SRS 完成
M3 设计完成	第 3 周结束	设计文档和编码规范完成
M4 数据与特征主线完成	第 5 周结束	数据链路、标签构造和相关测试可运行
M5 系统集成完成	第 7 周结束	训练、评估、可视化和确认测试可运行
M6 最终交付	第 8 周结束	所有文档、PDF、代码和演示材料提交完毕

1.15 13. 结论

本项目管理计划的核心不是追求复杂流程，而是在课程周期内确保目标明确、节奏稳定、责任清楚、交付可控。后续中期检查报告、测试计划和最终交付均以本计划中的时间线和里程碑为准。